

Edição de Aportes/Investimentos Implementada

Data: 02/11/2025

Autor: Sistema

Status:  Implementado e Testado

Objetivo

Adicionar funcionalidade de **edição de aportes** na seção de investimentos, permitindo que o usuário corrija ou atualize informações de investimentos já cadastrados, mantendo consistência com a edição de transações normais.

Funcionalidades Implementadas

1. Botão de Editar na Lista de Aportes

Interface

-  Ícone `Edit2` (lápiz) ao lado do botão de excluir
-  Cor verde `#00bf63` para destacar ação de edição
-  Aparece apenas no hover do card (transição suave)
-  Tooltip “Editar aporte” para melhor UX

Localização

```
// Localizado em: components/investments/investments-client.tsx
// Dentro do map de investments.slice(0, 10)
<Button
  variant="ghost"
  size="icon"
  onClick={() => handleEditInvestment(investment)}
  className="text-[#00bf63] hover:text-[#00a855] hover:bg-[#00bf63]/10 opacity-0
  group-hover:opacity-100 transition-opacity"
>
  <Edit2 className="h-4 w-4" />
</Button>
```

2. Dialog Dinâmico (Criar/Editar)

Modo de Criação

- Título: “**Novo Aporte**”
- Descrição: “Registre um novo investimento ou aporte”
- Botão: “**Adicionar Aporte**”
- Estado loading: “Adicionando...”

Modo de Edição

- Título: **“Editar Aporte”**
- Descrição: “Atualize as informações do seu aporte”
- Botão: **“Atualizar Aporte”**
- Estado loading: “Atualizando...”

Campos Editáveis

1. **Descrição** - Nome do investimento (ex: “CDB Banco XYZ”)
2. **Valor (R\$)** - Valor monetário do aporte
3. **Categoria** - Tipo de investimento (Renda Fixa, Ações, etc)
4. **Data** - Data do aporte
5. **Vincular à Meta** - Meta financeira associada (opcional)

3. Lógica de Atualização Inteligente

Atualização de Progresso de Metas

A funcionalidade implementa lógica complexa para manter a integridade das metas:

Cenário 1: Mudança de Meta

Meta Antiga: Reserva de Emergência (R\$ 5.000 investidos)
Meta Nova: Viagem (R\$ 2.000 investidos)
Aporte editado: R\$ 1.000 → R\$ 1.500

Resultado:

- Reserva de Emergência: $R\$ 5.000 - R\$ 1.000 = R\$ 4.000$ ✓
- Viagem: $R\$ 2.000 + R\$ 1.500 = R\$ 3.500$ ✓

Cenário 2: Mesma Meta, Valor Diferente

Meta: Aposentadoria (R\$ 10.000 investidos)
Aporte editado: R\$ 2.000 → R\$ 3.000

Resultado:

- Aposentadoria: $R\$ 10.000 - R\$ 2.000 + R\$ 3.000 = R\$ 11.000$ ✓
- Diferença aplicada: +R\$ 1.000

Cenário 3: Remover de Meta

Meta Antiga: Educação (R\$ 8.000 investidos)
Meta Nova: Sem meta

Resultado:

- Educação: $R\$ 8.000 - R\$ 2.000 = R\$ 6.000$ ✓
 - Aporte agora é independente
-

4. API Route PUT

Endpoint

```
PUT /api/investments
```

Payload

```
{
  "id": "cuid_do_investimento",
  "name": "Tesouro Selic 2027",
  "amount": 5000.00,
  "category": "Renda Fixa",
  "date": "2025-11-02",
  "goalId": "cuid_da_meta" ou null,
  "oldAmount": 4000.00,
  "oldGoalId": "cuid_da_meta_antiga" ou null
}
```

Validações

- ✓ Autenticação do usuário (NextAuth session)
- ✓ Verificação de propriedade do investimento
- ✓ Validação de campos obrigatórios
- ✓ Proteção contra edição de investimentos de outros usuários

Resposta de Sucesso (200)

```
{
  "id": "cuid",
  "name": "Tesouro Selic 2027",
  "amount": 5000,
  "category": "Renda Fixa",
  "date": "2025-11-02T00:00:00.000Z",
  "goalId": "cuid_da_meta",
  "goal": {
    "id": "cuid_da_meta",
    "name": "Reserva de Emergência",
    "targetAmount": 20000,
    "progress": 12500
  }
}
```

Implementação Técnica

Arquivos Modificados

1. components/investments/investments-client.tsx

Estados Adicionados:

```
const [editingInvestment, setEditingInvestment] = useState<Investment | null>(null)
```

Funções Implementadas:

```
// Abrir modal de edição e preencher formulário
handleEditInvestment(investment: Investment)

// Limpar formulário ao fechar
clearInvestmentForm()

// Criar novo aporte
handleCreateInvestment()

// Atualizar aporte existente
handleUpdateInvestment()

// Salvar (delegador - criar ou atualizar)
handleSaveInvestment()
```

2. app/api/investments/route.ts

Nova Rota:

```
export async function PUT(request: NextRequest) {
  // 1. Validar sessão
  // 2. Validar payload
  // 3. Verificar propriedade
  // 4. Atualizar investimento
  // 5. Ajustar progresso das metas:
  //   - Remover da meta antiga (se houver)
  //   - Adicionar na meta nova (se houver)
  //   - Ou ajustar diferença na mesma meta
  // 6. Retornar investimento serializado
}
```



Consistência de UX

Padrões Seguidos

1. Visual Idêntico às Transações

- Mesmo estilo de botões (editar verde, excluir vermelho)
- Mesma disposição (editar à esquerda, excluir à direita)
- Mesmos tooltips e feedback visual

2. Feedback ao Usuário

-  Toast de sucesso: "Aporte atualizado com sucesso! 
-  Toast de erro: "Erro ao atualizar investimento"
-  Loading states claros e descritivos

3. Prevenção de Erros

- Validação de campos obrigatórios
- Limpeza automática do formulário ao fechar
- Confirmação antes de ações destrutivas



Casos de Uso

Caso 1: Correção de Valor

Usuário:

1. Digita R\$ 1.000 em vez de R\$ 10.000
2. Clica em editar no aporte
3. Corrige o valor para R\$ 10.000
4. Clica em "Atualizar Aporte"

Sistema:

- Atualiza o aporte
- Ajusta progresso da meta (+R\$ 9.000)
- Exibe toast de sucesso
- Atualiza lista de aportes e metas

Caso 2: Mudança de Categoria

Usuário:

1. Cadastra aporte como "Ações"
2. Percebe que ☒ "Fundos"
3. Edita categoria para "Fundos"

Sistema:

- Atualiza categoria
- Mantém valor e meta inalterados
- Atualiza gráfico de distribuição

Caso 3: Vincular a Meta Posteriormente

Usuário:

1. Cria aporte sem vincular a meta
2. Depois cria meta "Viagem"
3. Edita aporte e vincula ☒ meta

Sistema:

- Vincula aporte ☒ meta
- Adiciona valor ao progresso da meta
- Atualiza cartões de meta



Testes Realizados

Build Local

- ✓ TypeScript compilation (yarn tsc)
- ✓ Next.js build (yarn build)
- ✓ Sem erros de tipos
- ✓ Sem erros de build

Validações

- ☒ Dialog abre corretamente com dados preenchidos
- ☒ Campos mantêm valores ao editar

- ☒ API retorna dados atualizados
- ☒ Progresso de metas recalculado corretamente
- ☒ Lista de aportes atualiza em tempo real
- ☒ Formulário limpa ao fechar dialog
- ☒ Tooltips aparecem no hover
- ☒ Estados de loading funcionam

Impacto

Antes

- ✗ Usuário precisava excluir e recriar aporte para corrigir erro
- ✗ Perda de histórico ao corrigir valores
- ✗ Impacto negativo no progresso das metas

Depois

- ☒ Edição direta e rápida
- ☒ Histórico preservado
- ☒ Ajuste automático e inteligente das metas
- ☒ UX consistente com o resto da aplicação
- ☒ Menos cliques para correções

Referências

Arquivos

- `/components/investments/investments-client.tsx` - Componente cliente
- `/app/api/investments/route.ts` - API de investimentos
- `/prisma/schema.prisma` - Schema do banco

Rotas Relacionadas

- `GET /api/investments` - Listar investimentos
- `POST /api/investments` - Criar investimento
- `PUT /api/investments` - ✨ **Novo: Editar investimento**
- `DELETE /api/investments` - Remover investimento

Lições Aprendidas

1. Consistência é Fundamental

- Seguir padrões existentes facilita aprendizado do usuário
- Reaproveitar componentes e estilos poupa tempo

2. Lógica de Negócio Complexa

- Atualização de metas requer cuidado com estados anteriores
- Importante passar `oldAmount` e `oldGoalId` para cálculos corretos

3. Feedback Visual Essencial

- Tooltips melhoram descoberta de funcionalidades
- Estados de loading reduzem ansiedade do usuário

Status Final:  Funcionalidade completa e em produção

Próximos Passos: Monitorar uso e feedback dos usuários